# Under What Conditions Is Encrypted Key Exchange Actually Secure?

Jake Januzelli[1(✉)], Lawrence Roy[2], and Jiayu Xu[1]

[1] Oregon State University, Corvallis, USA
{januzelj,xujiay}@oregonstate.edu
[2] Aarhus University, Aarhus, Denmark

**Abstract.** A Password-Authenticated Key Exchange (PAKE) protocol allows two parties to agree upon a cryptographic key, in the setting where the only secret shared in advance is a low-entropy password. The standard security notion for PAKE is in the Universal Composability (UC) framework. In recent years there have been a large number of works analyzing the UC-security of Encrypted Key Exchange (EKE), the very first PAKE protocol, and its One-encryption variant (OEKE), both of which compile an unauthenticated Key Agreement (KA) protocol into a PAKE.

In this work, we present a comprehensive and thorough study of the UC-security of both EKE and OEKE in the *most general* setting and using the *most efficient* building blocks:

1. We show that among the five existing results on the UC-security of (O)EKE using a general KA protocol, all are incorrect;
2. We show that for (O)EKE to be UC-secure, the underlying KA protocol needs to satisfy several additional security properties: though some of these are closely related to existing security properties, some are new, and all are missing from existing works on (O)EKE;
3. We give UC-security proofs for EKE and OEKE using Programmable-Once Public Function (POPF), which is the most efficient instantiation to date and is around 4 times faster than the standard instantiation using Ideal Cipher (IC).

Our results in particular allow for PAKE constructions from post-quantum KA protocols such as Kyber. We also present a security analysis of POPF using a new, weakened notion of *almost UC* realizing a functionality, that is still sufficient for proving composed protocols to be fully UC-secure.

## 1 Introduction

A *Password-Authenticated Key Exchange (PAKE)* protocol allows two parties to agree upon a cryptographic key in the setting where the only information shared in advance is a low-entropy password. Crucially, such protocols must be secure against man-in-the-middle adversaries that can arbitrarily modify the protocol messages sent between the two parties. PAKE protocols—and their extensions


S. Fehr and P.-A. Fouque (Eds.): EUROCRYPT 2025, LNCS 15602, pp. 451–481, 2025.
https://doi.org/10.1007/978-3-031-91124-8_16

such as asymmetric PAKE and threshold PAKE—provide significant advantages over the traditional "password-over-TLS" approach to authentication, in that PAKE does not require transmission of public keys and thus does not rely on PKI. Recent years have witnessed an increasing amount of interest in PAKE from both academia and industry, and with the recent standardization process by the IETF [Cry20], practical implementations—in particular, integration of PAKE protocols and TLS—is under active consideration [DFG+23].

*Security Notions for PAKE.* Since passwords have low entropy, an inevitable attack that must be taken into account by any security definition for PAKE is *online guessing*, where the adversary guesses a password $\mathsf{pw}^*$ and executes Alice's algorithm on $\mathsf{pw}^*$ while communicating with Bob; if $\mathsf{pw}^*$ is indeed Bob's password, then the adversary learns Bob's session key. The game-based security notion for PAKE [BPR00] requires that online guessing is the best possible attack; if the password is uniformly drawn from the dictionary $\mathsf{Dict}$, then the adversary's advantage is at most negligibly greater than $q/|\mathsf{Dict}|$ for $q$ online sessions. While this notion achieves a basic level of PAKE security, it comes with two significant drawbacks: First, it does not provide any security guarantee under parallel composition. This is especially devastating for PAKE, since (1) applications of PAKE generally involve a large number of users running the protocol in parallel, and (2) PAKE is usually composed with some symmetric-key primitives and rarely used in a standalone manner. Second, it fails to model password reuse across multiple accounts, which is (unfortunately) all too common in real life (Fig. 1).

For these reasons, the game-based PAKE security definition has been superseded by the definition in the Universal Composability (UC) framework [CHK+05], where security remains under arbitrary composition of protocols. Password reuse is addressed by letting the environment choose the password, rather than assuming a specific distribution over the dictionary. Over the years the UC notion has become the de facto security standard for PAKE; for example, all candidates in the second round of the IETF standardization competition have a UC security analysis [AHH21, ABB+20, JKX18, HL19].

*Encrypted Key Exchange (EKE).* The very first PAKE protocol ever proposed is *Encrypted Key Exchange (EKE)* by Bellovin and Merritt in 1992 [BM92]. It compiles any unauthenticated Key Agreement (KA) protocol into a PAKE by encrypting all messages using a private-key encryption scheme with the password as the key (and the receiver can decrypt and recover the message in the underlying KA protocol if it holds the correct password). In the standard instantiation of EKE, the encryption scheme is Ideal Cipher (IC). When instantiated with a 2-round KA protocol, we immediately obtain a PAKE protocol that also has 2 rounds.[1]

[1] The term "round" has various definitions in the literature. Here by 2-round we mean 2-flow, i.e., $P$ sends a message to $P'$, and $P'$, upon receiving the message from $P$, sends a message back. If the two messages can be sent simultaneously, we call the protocol 1-simultaneous round.

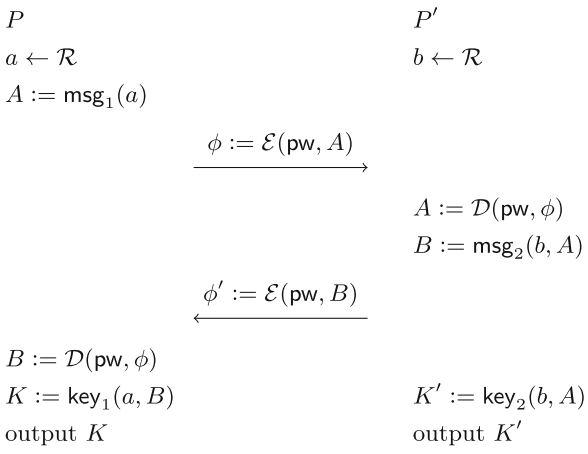


**Fig. 1.** EKE with a 2-round KA protocol

Aside from its historical significance, EKE continues to play an important role in PAKE design. Protocols such as SPAKE1 and SPAKE2 [AP05] are essentially EKE instantiated with specific private-key encryption schemes and KA protocols, and others such as CPace [HL19] inherit some design ideas from EKE. It is fair to say that EKE and its variants form one of the two major paradigms for PAKE protocols (the other is Smooth Projective Hash Function (SPHF)-based PAKEs [GL03,GK10,KV11] which are generally less efficient) (Fig. 2).

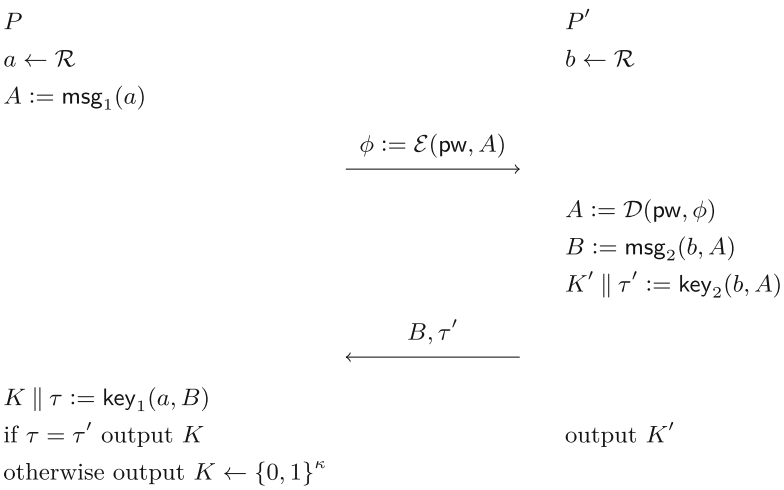


**Fig. 2.** OEKE with a 2-round KA protocol. This version requires a KA protocol with long key. In OEKE-PRF the KA key $K$ has length $\kappa$, PAKE session key is $\mathsf{PRF}_K(0)$, and $\tau = \mathsf{PRF}_K(1)$. In OEKE-RO $\mathsf{PRF}_K(x) = H(K, x)$

*One-Encryption EKE (OEKE).* The following variant of EKE first appeared in [BCP03]: only the $P$-to-$P'$ message is encrypted, and upon receiving the message from $P$, $P'$ sends a *plaintext* KA message to $P$ together with an authenticator that is the second part of $K'$ (where $K'$ is KA key of $P'$); the session key of $P'$ is the first part of $K'$.[2] $P$ computes its KA key $K$ and checks if the authenticator is the second part of $K$, and if so, outputs the first part of $K$ as its session key. This protocol is called OEKE (O for "One-encryption").

OEKE requires that the underlying KA protocol have key length longer than PAKE session key length $\kappa$. If we only have a KA protocol whose key $K$ is $\kappa$-bit long, one way to implement OEKE is to define the session key as $\mathsf{PRF}_K(0)$ and the authenticator as $\mathsf{PRF}_K(1)$ (where $\mathsf{PRF}$ is a PRF with $\kappa$-bit outputs)—that is, we compile the KA protocol into another KA protocol whose key is $\mathsf{PRF}_K(0) \,\|\, \mathsf{PRF}_K(1)$, and use the latter in OEKE. We call this protocol OEKE-PRF, and if the PRF is defined as $\mathsf{PRF}_K(x) = H(K, x)$ (where $H$ is an RO), we call it OEKE-RO. All existing works on the UC-security of OEKE follow the OEKE-PRF paradigm [SGJ23, BCP+23].[3]

While OEKE is not as round-efficient as EKE if the underlying KA protocol is 1-simultaneous round (such as Diffie-Hellman), it avoids one instance of IC and is thus more efficient in computation cost than EKE.

### 1.1 Existing Security Analyses of (O)EKE

Despite its deceiving simplicity and its pivotal role in over 30 years of study of PAKE, a satisfactory formal security analysis of EKE has long been elusive. The original EKE paper in 1992 [BM92] does not provide a security proof (in fact, even a formal security definition did not exist until 2000 [BPR00]). [BPR00], in addition to its main contribution of proposing the game-based security definition for PAKE, proves that EKE satisfies this definition under the Computational Diffie-Hellman (CDH) assumption if the underlying KA protocol is hashed Diffie-Hellman and the underlying private-key encryption scheme is IC. However, this result suffers from three drawbacks:

– The result assumes a specific KA protocol (hashed Diffie-Hellman) is used in EKE, and it is not immediately clear which properties one should require of

[2] Since each party has an output key in the KA protocol and an output key in the PAKE protocol (the latter being its eventual output), it might be confusing what a "key" refers to. Throughout this work we use the terms "KA key" and "(PAKE) session key" to distinguish them.

[3] Formally, [SGJ23, BCP+23] use a Key Encapsulation Mechanism (KEM) rather than a KA protocol as a building block, but a KEM is equivalent to a 2-round KA protocol: the first KA message is the KEM public key, the second KA message is the KEM ciphertext, and the KA key is the KEM key. (See, e.g., [SGJ23, Section 2.2].) [SGJ23] calls OEKE "EKE-KEM".

an arbitrary KA protocol for EKE to be secure. Pinpointing such properties is especially beneficial if we want to use a post-quantum KA protocol to achieve a PAKE under post-quantum assumptions—which was not a main concern when [BPR00] was published but has gained much importance and attention since then.
- The result assumes that the underlying private-key encryption scheme is IC. While it is known that an 8-round Feistel network is indifferentiable from an IC in the Random Oracle Model (ROM) [DS16], using an IC in EKE results in a PAKE protocol with significant cost. In particular, since the KA protocol is supposed to be hashed Diffie-Hellman, an IC *onto a group* is needed, which in turn requires 4 RO-hash onto a group operations which are inefficient.
- Finally, the result is in the game-based setting, which as we have mentioned has some critical drawbacks and has been superseded by the UC definition.

[BCP03] provides a security proof for OEKE, which suffers from the same drawbacks.

*UC Analyses of (O)EKE.* It was not until 18 years later that EKE was formally proven secure in the UC framework [DHP+18]. (For OEKE, we had to wait 20 years [SGJ23, BCP+23].) Since then, there have been a number of UC analyses of (O)EKE, each with its own instantiation. Below we give a brief summary of existing works in this domain:

- [MRR20, Theorem 10] claims that EKE is UC-secure provided that the underlying KA protocol is secure and *pseudorandom* (the KA messages are indistinguishable from uniformly random) and the underlying encryption scheme is *Programmable-Once Public Function (POPF)*, which can be seen as a generalization of IC. [MRR20] introduces the necessary (game-based) properties of the POPF for EKE to be UC-secure. Using a POPF instead of an IC drastically reduces costs, as a POPF can be instantiated by a 2-round Feistel network (as opposed to 8-round in IC). Furthermore, since [MRR20] uses *any* secure and pseudorandom KA protocol, it allows for an instantiation of EKE under post-quantum assumptions. Unfortunately, as we will see below, the main result in [MRR20] is incorrect in three different ways.
- [SGJ23, Theorem 2] also claims UC-security of EKE assuming the underlying KA protocol is secure and pseudorandom, and the underlying encryption scheme is *Half Ideal Cipher (HIC)*. [SGJ23] proposes a UC definition for HIC and shows that it is realized by a 2-round Feistel network, except that the hash-and-XOR operation in the second round is replaced by an IC *over* $2\kappa$*-bit strings*. Thus, while HIC (unlike POPF) requires an IC, it achieves UC-security and is easier to use in other contexts. However, we will show that this EKE result is incorrect in two different ways. In addition to the result above, [SGJ23, Theorem 3] claims that a certain variant of OEKE is UC-secure, again using HIC and a secure and pseudorandom KA protocol. Unfortunately, it is not entirely clear whether the protocol analyzed in [SGJ23] is OEKE-PRF or OEKE-RO, and there seems to be a mismatch between the theorem statement and its proof, as mentioned in [SGJ23, Footnote 14]:

> *The proof below assumes a version of the protocol which uses* $\mathsf{prf}(K, \cdot)$ *to derive the authenticator* $\tau$ *and the session key [...] This version of the protocol requires an additional assumption on KEM. We will update the proof shortly to reflect the modified protocol and get rid of the additional assumption.*

Using our terminology, the statement of [SGJ23, Theorem 3] claims the UC-security of OEKE-RO, while its proof is about the UC-security of OEKE-PRF; the latter requires an additional assumption on the underlying KA protocol (which is overlooked in the proof). However, it is never specified what this "additional assumption" is! As such, at least the security proof of [SGJ23, Theorem 3] seems incomplete.[4] There is also a separate issue that renders this result incorrect no matter whether OEKE-PRF or OEKE-RO is considered.

– [BCP+23, Theorem 1] proves the UC-security of EKE using IC and a secure and pseudorandom KA protocol, with the session key being an RO hash of the KA key together with the PAKE transcript. This result has a flavor similar to [MRR20, Theorem 10] and [SGJ23, Theorem 2], but is less efficient (it uses IC rather than POPF or HIC). Close scrutiny shows that this result is also incorrect: we will give two actual attacks that break the UC-security in different ways, and also show a flaw in a reduction in the hybrid proof that results in an incorrect bound on the tightness of the security.
[BCP+23, Theorem 2] proves the UC-security of OEKE-RO using IC and a secure and pseudorandom KA protocol, with the session key being an RO hash of the KA key together with the PAKE transcript. We will show that this security statement is also incorrect.

There are several additional analyses of EKE with *specific* KA protocols: [DHP+18, Theorem 6] shows that EKE using hashed Diffie-Hellman is UC-secure under CDH, and [LLHG23, Theorem 2] shows that EKE using hashed *twin* Diffie-Hellman (where one party sends two group elements $g^{a_1}$ and $g^{a_2}$ and the other sends a single group element $g^b$, and the KA key is $H(g^{a_1 b}, g^{a_2 b})$) is *tightly* UC-secure under CDH. While both security statements are correct, we will see that the proof of [DHP+18, Theorem 6] contains a flawed reduction in the hybrids very similar to the tightness issue in [BCP+23, Theorem 1]. Finally, the concurrent and independent work of [ABJS24] considers OEKE with *splittable* underlying KA protocols, meaning that the first KA message can be encoded as a bitstring and a group element.

See Table 1 for a summary of existing UC security analyses of (O)EKE.

We can see that despite a large amount of works analyzing the UC-security of (O)EKE in various flavors, all existing analyses using a general KA protocol are incorrect. This brings us to the question we ask in the title:

*Under what conditions is EKE (and its variants) actually secure?*

[4] Looking ahead, this "additional assumption" is what we call *collision resistance* for the underlying KA protocol (see Sect. 3.1).

### 1.2 Our Contributions

In this work, we present a comprehensive study of the UC-security of EKE and its one-encryption variant. We consider the protocols instantiated with the *most efficient* encryption scheme, i.e., POPF; and the *most general* KA protocols, which allows us to obtain a wide range of PAKE protocols, including ones based on post-quantum assumptions. Concretely, our contributions are as follows:

**Table 1.** UC security analyses of (O)EKE. Flawed analyses are marked in grey

| result | protocol analyzed | KA protocol | encryption scheme | note |
|---|---|---|---|---|
| [DHP+18, Theorem 6] | EKE | hashed Diffie-Hellman (PAKE transcript included in hash) | IC | proof flawed |
| [MRR20, Theorem 10] | EKE | general | POPF | result incorrect (Sects. 3.2 and 3.3) |
| [SGJ23, Theorem 2] | EKE | general | HIC | result incorrect (Sect. 3.2) |
| [SGJ23, Theorem 3] | OEKE-? | general | HIC | result ambiguous, unclear if OEKE-PRF or OEKE-RO; security proof incorrect (Sect. 3.1); either way result also incorrect |
| [LLHG23, Theorem 2] | EKE | hashed twin Diffie-Hellman (password included in hash) | IC | |
| [BCP+23, Theorem 1] | EKE | general (PAKE transcript included in hash) | IC | result incorrect (Sect. 3.3) |
| [BCP+23, Theorem 2] | OEKE-RO | general (PAKE transcript included in hash) | IC | result incorrect |
| our Thm. 5.1 | EKE | general | POPF | |
| our Thm. 5.2 | OEKE | general | POPF | |

*Attacks on Previous Instantiations of (O)EKE.* In Sect. 3 we motivate our KA security properties with several attacks:

– In Sect. 3.1 we show that OEKE (including its PRF and RO variants) is insecure if the underlying KA protocol does not satisfy *collision resistance* (Def. 3.1). This implies that the proof of [SGJ23, Theorem 3] is incorrect;

- In Sect. 3.2 we show that EKE is insecure if the underlying KA protocol does not satisfy *pseudorandom non-malleability* (Def. 3.2), which implies that [MRR20, Theorem 10] and [SGJ23, Theorem 2] are false;
- In Sect. 3.3 we show that EKE is insecure if the underlying KA protocol does not satisfy *strong pseudorandomness* (Def. 2.5), which implies that [MRR20, Theorem 10] and [BCP+23, Theorem 1] are false.

Although some of these KA notions are related to existing security properties in the Key-Encapsulation Mechanism (KEM) literature, to the best of our knowledge we are the first to formalize strong pseudorandomness and the first to notice the necessity of these properties for the security of (O)EKE. In the full version [JRX24] we present several additional attacks and flaws in existing works; in particular, we show pseudorandom non-malleability is also needed in OEKE, and present a further attack that only applies to EKE using HIC or POPF (instead of IC).

*POPF and "almost UC".* The results in Sect. 3 are sufficient to recover correct security proofs of (O)EKE using IC as the encryption mechanism (those proofs are sketched in [JRX24]). However, the most efficient instantiation of (O)EKE uses POPF as defined by [MRR20], so as an additional contribution we give our security proofs in the POPF setting. At a high level, a POPF is a function family $\{F_\phi\}$ where each $F_\phi$ is random everywhere, except that when generating $\phi$ one can program a single input/output pair $(x, y)$ such that $F_\phi(x) = y$. To capture this notion of "uncontrollable outputs", it is required that for any weak pseudorandom function $W_k$, $W_k(F_\phi(x'))$ is pseudorandom for $x' \neq x$. A POPF can be viewed as a randomized version of IC, where programming $F_\phi(x)$ to be $y$ is analogous to encrypting $y$ under key $x$ (resulting in ciphertext $\phi$), and evaluating $F_\phi$ at $x$ is analogous to decrypting the ciphertext $\phi$ with key $x$. However, in the case of POPF there can be many $\phi$ such that $F_\phi(x) = y$, whereas for IC there is only a single $\phi$ such that $\mathcal{E}(x, y) = \phi$. Although [MRR20] used game-based security properties for POPF in the analysis of EKE, they separate honest simulation and extraction [MRR20, Definitions 7.2-7.3], which is insufficient to analyze the case of a man-in-the-middle adversary where the simulator may have to do both simultaneously.

An obvious strategy to recover a proof of security is to define a UC functionality for POPF, and then use the UC POPF functionality in (O)EKE. Unfortunately, the POPF in [MRR20] does not seem to realize an appropriate UC functionality. The problem is the proof that $\{F_\phi\}$ has "uncontrollable outputs" reduces to the security of the weak PRF $W_k$, during which the non-tight reduction needs to make a guess over the adversary's random oracle queries. In other words, we reduce to the security of some schemes built *on top* of POPF, not to an *underlying assumption*. If we attempt to show the POPF UC-realizes a functionality, the simulator must make the RO behavior consistent with the POPF functionality, which requires it to guess the RO query, making the simulator fail. Crucially, while a loose *reduction* is sufficient to prove security, a UC *simulator* must correctly simulate the real world with overwhelming probability, as the

environment could otherwise tell that it is in the ideal world with significant probability.

We solve this by showing that POPF "almost UC-realizes" a functionality $\mathcal{F}_{\text{POPF}}$, where instead of full UC-realization of a functionality, we only require that *a specific class* of higher-level protocols that use $\mathcal{F}_{\text{POPF}}$ remain secure if the real POPF is used instead.[5] We are able to achieve this weaker security notion, as now the guess of an RO query can appear in a *hybrid* in the proof of security of the higher-level protocol, which is not an issue. (A similar technique was used in [CDG+18].) At the same time, the almost UC notion is sufficient for the higher-level protocol; in particular, (O)EKE built on top of our POPF is (fully) UC-secure. We expect the notion of "almost UC-realization" to have applications beyond the present work. E.g, the security loss when replacing an ideal functionality with a protocol that UC-realizes it is additive by the UC composition theorem [Can01, Theorem 3]: almost-UC can be used to recover secure composition in cases where one expects a super-additive security loss.

*Analysis of (O)EKE Using POPF and Suitable KA Protocols.* Equipped with our almost UC modeling of POPF and our additional KA properties, we now turn to our main constructive contribution: an analysis of both EKE and OEKE in the UC framework, using a general KA protocol. There are a large number of existing analyses of (O)EKE that vary according to:

1. The procedure by which PAKE session keys are derived.
2. The properties the KA satisfies, other than the standard notions of correctness and security.
3. The encryption mechanism used, whether that be IC, HIC, or POPF.

In this work, we clarify exactly which choices from these three dimensions result in a secure protocol, along the way obtaining the most efficient instantiations of (O)EKE to date:

1. *"Minimal" output function*: We show that for EKE using POPF, the session key needs to be a PRF of the second PAKE message under the KA key (we call this protocol EKE-PRF), and no other items need to be included; if we use IC instead, outputting the "raw" KA key suffices (provided that the KA protocol satisfies appropriate properties—see below). For OEKE, we show that the "raw" version is secure, and no items need to be hashed (again, provided that the KA protocol satisfies appropriate properties); this in particular covers both the PRF and RO variants.
2. *Exact KA properties*: We show that EKE-PRF is secure if the underlying KA protocol satisfies *strong pseudorandomness* and *pseudorandom non-malleability*, in addition to correctness and security. Additionally, we show that OEKE is secure if the underlying KA protocol satisfies pseudorandom

[5] This idea of only requiring it to preserve the security of protocols built on top is similar to security-preserving samplers defined by [AWZ23]. The goal there is quite different however: the elimination of ROs from a 1-round protocol.

non-malleability and *collision resistance* in addition to correctness and security, covering both the OEKE-PRF and OEKE-RO variants. In each case, we give concrete attacks showing that these additional KA properties are necessary for (O)EKE to be secure, and are not implied by existing standard properties.

3. *Most efficient instantiations*: Our main results about both EKE and OEKE use POPF, which are the most efficient instantiations to date. This provides significant efficiency advantage over the traditional IC implementation, as POPF only requires 2 rounds of Feistel network (as opposed to 8 in IC). To compare, the HIC construction of [SGJ23] requires a Feistel rounds to use an IC over $2\kappa$-bit strings, while our construction only requires a random oracle. Though an IC over bitstrings is much less troublesome than an ideal cipher over curve points, it is still not trivial to instantiate—most block ciphers are designed for $\kappa$-bit block size, not $2\kappa$. Suitable choices will either be less efficient or less conservative than instantiating a random oracle with a standard hash function.

In addition to the theoretical advantages outlined above, our approach allows practitioners looking to implement (O)EKE to avoid analyzing the (O)EKE protocol as a whole, and instead merely verify whether their chosen KA protocol satisfies a few properties, which is much easier.

## 2 Preliminaries

*UC Conventions.* We assume w.l.o.g. that the session ID always includes the names of the two parties. Furthermore, for ideal objects such as RO and IC, we always assume that the session ID $\mathsf{sid}$ is part of the input (for IC, it is part of the key, i.e., a ciphertext is in the form of $\mathcal{E}(\mathsf{sid} \parallel k, m)$) and do not explicitly write them. This is for brevity only; we stress that in actual protocols the session ID should be included to achieve domain separation.

### 2.1 (Unauthenticated) Key Agreement Protocols

A 2-round *Key Agreement (KA)* protocol (henceforth KA protocol) consists of the following (deterministic) algorithms:

- $\mathsf{msg}_1(a) = A \in \mathcal{M}_1$: protocol-message function for party $P$
- $\mathsf{msg}_2(b, A) = B \in \mathcal{M}_2$: protocol-message function for party $P'$
- $\mathsf{key}_1(a, B) = K \in \mathcal{K}$: output function for party $P$
- $\mathsf{key}_2(b, A) = K' \in \mathcal{K}$: output function for party $P'$

In a standard execution of the protocol, party $P$ samples randomness $a \leftarrow \mathcal{R}$ and sends $A := \mathsf{msg}_1(a)$ to party $P'$. $P'$ then samples $b \leftarrow \mathcal{R}$, sends $B := \mathsf{msg}_2(b, A)$ to $P$, and outputs $K' := \mathsf{key}_2(b, A)$. Upon receiving $B$, $P$ outputs $K := \mathsf{key}_1(a, B)$.

**Definition 2.1.** A KA protocol is **correct** if

$$\mathsf{key}_1(a, \mathsf{msg}_2(b, \mathsf{msg}_1(a))) = \mathsf{key}_2(b, \mathsf{msg}_1(a))$$

with overwhelming probability when $a, b \leftarrow \mathcal{R}$.

*Security.*

**Definition 2.2.** A KA protocol is **secure** if the following two distributions are indistinguishable:

| | |
|---|---|
| $a \leftarrow \mathcal{R}$ | $a \leftarrow \mathcal{R}$ |
| $b \leftarrow \mathcal{R}$ | $b \leftarrow \mathcal{R}$ |
| $A := \mathsf{msg}_1(a)$ | $A := \mathsf{msg}_1(a)$ |
| $B := \mathsf{msg}_2(b, A)$ | $B := \mathsf{msg}_2(b, A)$ |
| $K := \mathsf{key}_1(a, B)$ | $K \leftarrow \mathcal{K}$ |
| output $(A, B, K)$ | output $(A, B, K)$ |

*Pseudorandomness.* We also consider the notion of pseudorandomness in which the protocol messages are also indistinguishable from random.

**Definition 2.3.** A KA protocol has **pseudorandom first message**, or **first pseudorandomness**, if the following two distributions are indistinguishable:

| | |
|---|---|
| $a \leftarrow \mathcal{R}$ | |
| $A := \mathsf{msg}_1(a)$ | $A \leftarrow \mathcal{M}_1$ |
| output $A$ | output $A$ |

**Definition 2.4.** A KA protocol has **pseudorandom second message**, or **second pseudorandomness**, if the following two distributions are indistinguishable:

| | |
|---|---|
| $a \leftarrow \mathcal{R}$ | $a \leftarrow \mathcal{R}$ |
| $A := \mathsf{msg}_1(a)$ | $A := \mathsf{msg}_1(a)$ |
| $b \leftarrow \mathcal{R}$ | |
| $B := \mathsf{msg}_2(b, A)$ | $B \leftarrow \mathcal{M}_2$ |
| output $(A, B)$ | output $(A, B)$ |

The two pseudorandomness properties combined imply that the *joint distribution* of the two KA messages are indistinguishable from random.

*Strong Pseudorandomness.* In the security analysis for EKE we need a stronger pseudorandomness property, which says that the second message is pseudorandom *even given the key derivation function used on it.*

**Definition 2.5.** A KA protocol has **strong pseudorandom second message**, or **strong second pseudorandomness**, if the following two distributions are indistinguishable:

| | |
|---|---|
| $a \leftarrow \mathcal{R}$ | $a \leftarrow \mathcal{R}$ |
| $A := \mathsf{msg}_1(a)$ | $A := \mathsf{msg}_1(a)$ |
| $b \leftarrow \mathcal{R}$ | |
| $B := \mathsf{msg}_2(b, A)$ | $B \leftarrow \mathcal{M}_2$ |
| $K := \mathsf{key}_1(a, B)$ | $K := \mathsf{key}_1(a, B)$ |
| output $(A, B, K)$ | output $(A, B, K)$ |

Henceforth we may call the combination of first and second pseudorandomness *pseudorandomness*, and the combination of first and strong second pseudorandomness *strong pseudorandomness*.

To our knowledge, strong pseudorandomness does not directly correspond to any existing KA/KEM security notion, and as such has not appeared in any existing works on EKE that use general 2-round KA. In Sect. 3.3 we show strong pseudorandomness is necessary for the security of EKE (but not OEKE) and is not implied by the standard notions of security and pseudorandomness. However, a KEM that satisfies both SPR-CCA$_{\mathcal{S}}$ *and* SMT-CCA$_{\mathcal{S}}$ security from [Xag22] also satisfies strong pseudorandomness by a simple hybrid argument.

- On input $(\mathsf{NewSession}, \mathsf{sid}, P, P', \mathsf{pw}, \mathsf{role})$ from $P$, send $(\mathsf{NewSession}, \mathsf{sid}, P, P', \mathsf{role})$ to $\mathcal{S}$. Furthermore, if this is the first NewSession message for *sid*, or this is the second NewSession message for sid and there is a record $\langle P', P, \cdot \rangle$, then record $\langle P, P', \mathsf{pw} \rangle$ and mark it fresh.
- On $(\mathsf{TestPwd}, \mathsf{sid}, P, \mathsf{pw}^*)$ from $\mathcal{S}$, if there is a record $\langle P, P', \mathsf{pw} \rangle$ marked fresh, then do:
  - If $\mathsf{pw}^* = \mathsf{pw}$, then mark the record compromised and send "correct guess" to $\mathcal{S}$.
  - If $\mathsf{pw}^* \neq \mathsf{pw}$, then mark the record interrupted and send "wrong guess" to $\mathcal{S}$.
- On $(\mathsf{NewKey}, \mathsf{sid}, P, K^* \in \{0,1\}^\kappa)$ from $\mathcal{S}$, if there is a record $\langle P, P', \mathsf{pw} \rangle$, and this is the first NewKey message for *sid* and $P$, then output $(\mathsf{sid}, K)$ to $P$, where $K$ is defined as follows:
  - If the record is compromised, or either $P$ or $P'$ is corrupted, then set $K := K^*$.
  - If the record is fresh, a key $(\mathsf{sid}, K')$ has been output to $P'$, at which time there was a record $\langle P', P, \mathsf{pw} \rangle$ marked fresh, then set $K := K'$.
  - Otherwise sample $K \leftarrow \{0,1\}^\kappa$.

  Finally, mark the record completed.

**Fig. 3.** UC PAKE functionality $\mathcal{F}_{\text{PAKE}}$

## 2.2 UC PAKE Functionality

We recall the standard UC PAKE functionality [CHK+05] in Fig. 3. For a detailed explanation of the functionality, see [RX23].

It is subtle and critical to our later sections how exactly a session key is determined, so let us explain the three cases under NewKey :

- The "normal" case is that both the $P$ session and the $P'$ session are fresh, and $\mathsf{pw} = \mathsf{pw}'$. This models a correct protocol execution in the real world, where the adversary does not interfere and the two parties' passwords match. Say the $P'$ session ends first; then $P'$ outputs a random session key $K'$ (the third case under NewKey), and when the $P$ session ends, $P$ outputs session key $K = K'$ (the second case).
- If both the $P$ session and the $P'$ session are fresh, but $\mathsf{pw} \neq \mathsf{pw}'$, then this corresponds to an incorrect protocol execution where the adversary does not interfere but the two parties' passwords do not match. In this case $P$ and $P'$ output independent random keys (the third case).
- If the $P$ session is compromised, it models the real-world scenario where the adversary has successfully performed an online guessing attack on $P$. In this case all security guarantees are lost, so we might as well let the adversary choose the session key for $P$ (the first case). The same goes for the $P'$ session (same below).
- If the $P$ session is interrupted, it models the real-world scenario where the adversary has performed an unsuccessful online guessing attack on $P$. This means that the adversary should have no information about the session key of $P$; furthermore, the session key of $P$ should also be independent of the session key of $P'$. So we let $P$ output a random session key (the third case).
- Finally, if the $P$ session is fresh but its counter session $P'$ has been attacked (either compromised or interrupted), then again the adversary should have no information about the session key of $P$ (because the $P$ session is fresh), and the session key of $P$ should also be independent of the session key of $P'$ (because the $P'$ session is attacked, so $P'$ should output a session key that is either set by the adversary or independent of everything else). So we also let $P$ output a random session key (the third case).

We stress that if a party's session key is random (the first three cases), *the adversary never gains any information about it*. For example, if the $P$ session is fresh and the $P'$ session is compromised, then the session key of $P$ is still independent of the adversary's view (even though the adversary fully controls the session key of $P'$). The above holds even if the adversary learns the party's password after the session ends with a random session key; in this case the session is already marked completed, so the adversary cannot send TestPwd even if it gets to know the password later on.

## 3 Attacks on Previous Instantiations of (O)EKE

In this section, we show that a number of (O)EKE instantiations, including the ones claimed secure in [MRR20, Theorem 10], [SGJ23, Theorem 2], the proof of [SGJ23, Theorem 3], and [BCP+23, Theorem 1], are actually insecure. For each attack we also describe the security property necessary to prevent it. We assume

that the encryption scheme used in (O)EKE is an IC; our attacks analogously apply to uses of POPF or HIC.

The attacks in Sects. 3.1 and 3.2 are presented with the underlying KA protocol being (some variants of) Diffie-Hellman. This is mainly for clarity of presentation, but we stress that (1) these attacks are sufficient to invalidate existing results on the security of (O)EKE, as (variants of) Diffie-Hellman satisfy all properties claimed to imply security yet the corresponding (O)EKE protocols are insecure, and (2) from these attacks we can see *in general* what type of additional properties the underlying KA protocol needs to satisfy. The attack in Sect. 3.3 assumes a general KA protocol.

The attack in Sect. 3.2 is taken from the talk [Jar23] for the paper [SGJ23]; however, the full version of [SGJ23] has not been updated accordingly after the talk, and to the best of our knowledge, we are the first to present this attack in written form. Furthermore, we formally prove that the protocol in question is *not* UC-secure. The other two attacks are our work.

### 3.1 Allowing Identity Element in Diffie-Hellman Makes OEKE Insecure

We begin with a simple attack on OEKE-PRF where the underlying KA protocol is plain Diffie-Hellman; that is, $P$ sends $\mathcal{E}(\mathsf{pw}, g^a)$ to $P'$, $P'$ samples integer $b$ and computes its KA key $K' = (g^a)^b = g^{ab}$ and PAKE session key $\mathsf{PRF}_{K'}(0)$, and finally sends $g^b$ together with $\tau' = \mathsf{PRF}_{K'}(1)$ to $P$. $P$ then computes $K = g^{ab}$ as $(g^b)^a$, and checks if $\tau' = \mathsf{PRF}_K(1)$ (if so, $P$ outputs $\mathsf{PRF}_K(0)$ as its session key; otherwise $P$ outputs a random session key).

Consider the following adversary: it disregards the $P$-to-$P'$ message and sends $(e, \mathsf{PRF}_e(1))$ to $P$, where $e$ is the identity group element. Then $P$ computes $K = e^a = e$, so the check passes and $P$ outputs session key $\mathsf{PRF}_K(0) = \mathsf{PRF}_e(0)$. That is, an adversary that does not know the password can predict the session key of $P$, which clearly breaks the security of PAKE. This attack can be easily generalized to any version of OEKE using plain Diffie-Hellman, and in particular OEKE-RO is also insecure; and it extends to OEKE using hashed Diffie-Hellman (the adversary sends $(e, \mathsf{PRF}_{H(e)}(1))$ to $P$). An obvious fix is to disallow $e$ as a KA message, i.e., sample the exponent $b$ from $\mathbb{Z}_p \setminus \{0\}$ rather than $\mathbb{Z}_p$. Note that the attack does not apply to EKE where the $P'$-to-$P$ message $g^b$ is encrypted under $\mathsf{pw}$ (so sending $\mathcal{E}(\mathsf{pw}, e)$ requires knowledge of $\mathsf{pw}$).

(For a diagram of the attack see [JRX24].)

The attack above shows that *any* KA protocol underlying OEKE must satisfy some form of *contributoryness* property, namely party 1 must "contribute" to the output key and party 2 cannot single-handedly bias the distribution of the key too much. We formalize the exact property necessary for the security of OEKE, which we call *collision resistance*. The proof of [SGJ23, Theorem 3] only requires the KA protocol to be secure and pseudorandom; in other words, it allows for using plain Diffie-Hellman—where $e$ can be a message—in OEKE, so this proof is incorrect. (The theorem statement assumes a slightly different version of OEKE which prevents the attack in this section; see [JRX24].)

**Definition 3.1.** A KA protocol is **collision-resistant** if the key space is $\mathcal{K} = \{0,1\}^{3\kappa}$, and for any polynomially bounded $q$ and any PPT adversary $\mathcal{A}$, the winning probability of $\mathcal{A}$ in the following game is negligible:

$$\boxed{\begin{array}{l}\forall i \in [q]\colon a_i \leftarrow \mathcal{R} \\ \forall i \in [q]\colon A_i := \mathsf{msg}_1(a_i) \\ B^* \leftarrow \mathcal{A}(A_1, \ldots, A_q) \\ \forall i \in [q]\colon K_i \parallel \tau_i := \mathsf{key}_1(a_i, B^*) \\ \mathcal{A} \text{ wins if } \exists_{i \neq j}\, \tau_i = \tau_j\end{array}}$$

Here, we split keys $\mathsf{key}_1(a_i, B^*) \in \{0,1\}^{3\kappa}$ into two chunks: $K \in \{0,1\}^{\kappa}$ and $\tau \in \{0,1\}^{2\kappa}$.[6]

**Remark 3.1.** A version of collision-resistance for KEMs appears in [Xag22] as SCFR-CCA security. Our definition differs from theirs in three ways:

1. We do not give the adversary access to a decryption oracle;
2. We give the adversary polynomially many first messages instead of two;
3. We only require partial key collision to win the security game.

### 3.2 EKE with Plain Diffie-Hellman Is Insecure

Our next attack considers the EKE instantiation where the underlying KA protocol is plain Diffie-Hellman; that is, $P$ sends $\mathcal{E}(\mathsf{pw}, g^a)$ to $P'$ and $P'$ sends $\mathcal{E}(\mathsf{pw}, g^b)$ to $P$, and both parties decrypt each other's message and agree upon the session key $g^{ab}$. This protocol is insecure due to the following man-in-the-middle attack: an adversary can pass the $P$-to-$P'$ message $\mathcal{E}(\mathsf{pw}, g^a)$ without modification, then guess $\mathsf{pw}$ and replace the $P'$-to-$P$ message $\mathcal{E}(\mathsf{pw}, g^b)$ with $\mathcal{E}(\mathsf{pw}, g^{2b})$ (by decrypting the message and obtaining $g^b$, then encrypting $(g^b)^2$ under $\mathsf{pw}$). Assuming the password guess is correct, $P$ outputs $K = g^{2ab}$ and $P'$ outputs $K' = g^{ab}$, so $K = (K')^2$. In other words, an adversary that does not attack the $P'$ session but successfully attacks the $P$ session, causes the session keys of $P$ and $P'$ to be correlated—which is not allowed in a UC-secure PAKE.

(For a diagram of the attack see [JRX24]).

Indeed, the UC PAKE functionality $\mathcal{F}_{\text{PAKE}}$ guarantees that if the $P'$ session is not attacked but its counter-session was (successfully or unsuccessfully) attacked, then the $P'$ session is $\mathsf{fresh}$ and *the session key of $P'$ is independent of everything else* (see the last bullet at the end of Sect. 2.2). This in particular means that the protocol above violates UC-security for PAKE: when the adversary $\mathcal{A}$ passes $\mathcal{E}(\mathsf{pw}, g^a)$ to $P'$, the UC simulator $\mathcal{S}$ does not know $\mathsf{pw}$ and has to let $P'$ output a session key by sending a $(\mathsf{NewKey}, \mathsf{sid}, P', \star)$ message to $\mathcal{F}_{\text{PAKE}}$, causing $P'$ to output a random session key $K'$ (independent of the view of $\mathcal{S}$). Later, when $\mathcal{A}$ sends $\mathcal{E}(\mathsf{pw}, g^{2b})$ to $P$, $\mathcal{S}$ can extract $\mathsf{pw}$ and compromise the $P$ session by

[6] We require $\tau$ to be $2\kappa$-bit long, as an adversary can win the experiment with probability roughly $q^2/2^{|\tau|}$.

sending a correct TestPwd message, but $K'$ is still independent of the view of $\mathcal{S}$, so $\mathcal{S}$ cannot make $K'$ and the session key of $P$ correlated.

It is not hard to turn the above observation into a formal proof of UC-insecurity, which can be found in [JRX24].

The above shows that [MRR20, Theorem 10] and [SGJ23, Theorem 2] are false, since both theorems only require security and pseudorandomness of the underlying KA protocol, which are satisfied by plain Diffie-Hellman.

*Necessity of Pseudorandom Non-malleability.* The issue above points to a property of the underlying KA protocol that is not commonly seen in the literature but is required for the security of EKE: Consider a "semi-man-in-the-middle" adversary that sees both protocol messages, but can only modify the second, i.e., the $P'$-to-$P$ message. Then as long as the adversary modifies the $P'$-to-$P$ message, the output key of $P'$ is pseudorandom even if the adversary additionally sees the output of $P$. We call this property *non-malleability*. In fact, we require a stronger security property we call *pseudorandom non-malleability*, which says that the $P'$-to-$P$ message and the key of $P'$ are both pseudorandom (see [JRX24] for why we require pseudorandom non-malleability instead of just non-malleability).

**Definition 3.2.** A KA protocol is **pseudorandom non-malleable** if the following two distributions are indistinguishable:

| | |
|---|---|
| $a \leftarrow \mathcal{R}$ | $a \leftarrow \mathcal{R}$ |
| $b \leftarrow \mathcal{R}$ | |
| $A := \mathsf{msg}_1(a)$ | $A := \mathsf{msg}_1(a)$ |
| $B := \mathsf{msg}_2(b, A)$ | $B \leftarrow \mathcal{M}_2$ |
| $K' := \mathsf{key}_2(b, A)$ | $K' \leftarrow \mathcal{K}$ |
| $B^* \leftarrow \mathcal{A}(A, B, K')$ | $B^* \leftarrow \mathcal{A}(A, B, K')$ |
| abort if $B^* = B$ | abort if $B^* = B$ |
| $K := \mathsf{key}_1(a, B^*)$ | $K := \mathsf{key}_1(a, B^*)$ |
| output $K$ to $\mathcal{A}$ | output $K$ to $\mathcal{A}$ |

**Remark 3.2.** Non-malleability alone is implied by the CCA-security of a KEM, but pseudorandom non-malleability corresponds roughly to the IND$-CCA security of a KEM. A version of pseudorandom non-malleability for KEMs appears in [Xag22] as SPR-CCA$_{\mathcal{S}}$ security. Our definition is the same as SPR-CCA$_{\mathcal{S}}$ when the simulator $\mathcal{S}$ samples a uniform random ciphertext, except that we only allow the adversary to make a single decryption oracle query.

### 3.3 EKE Is Insecure If the Underlying KA Is Not Strongly Pseudorandom

The two attacks in previous sections assume the (O)EKE protocol uses (some variants of) the Diffie-Hellman KA. In this section we present a general attack

showing that for EKE to be secure the KA protocol needs to be *strongly* pseudorandom. This is not an issue for Diffie-Hellman which has perfect pseudorandomness (i.e., the protocol messages are uniform), but in general strong pseudorandomness is not implied by security and pseudorandomness—as we will show next—and thus must be presented as a property of its own. Please see Defs. 2.2 to 2.5 for the definitions of these properties.

*Indistinguishability Between Six Distributions (but not the seventh).* In a KA protocol, each of the first message $A$, the second message $B$, and the key $K$ may be "real" or "random", resulting in 8 potential joint distributions of $(A, B, K)$. (See Table 2 for definitions of "real" and "random".) The case where $A, B$ are random but $K$ is real (henceforth (random $A$, random $B$, real $K$), or simply (random, random, real)) is not well-defined, since for $K$ to be real, either $a$ or $b$ needs to be defined (which implies that either $A$ or $B$ needs to be real). Are the remaining 7 distributions indistinguishable from each other?

**Table 2.** Definitions of "real" and "random" for $A, B, K$

| | real | random |
|---|---|---|
| $A$ | $a \leftarrow \mathcal{R}\ A := \mathsf{msg}_1(a)$ | $A \leftarrow \mathcal{M}_1$ |
| $B$ | $b \leftarrow \mathcal{R}\ B := \mathsf{msg}_2(b, A)$ | $B \leftarrow \mathcal{M}_2$ |
| $K$ | $K := \mathsf{key}_1(a, B)$ or $K := \mathsf{key}_2(b, A)$ | $K \leftarrow \mathcal{K}$ |

Recall that first pseudorandomness says that real $A$ is indistinguishable from random $A$, and second pseudorandomness says that (real $A$, real $B$) is indistinguishable from (real $A$, random $B$). Note that the definition of real $B$ does not use $a$ (it only uses $A$) and thus can be simulated by a reduction to first pseudorandomness; this means that pseudorandomness (first and second combined) implies that the 4 joint distributions of $A$ and $B$—where both $A$ and $B$ might be real or random—are indistinguishable from each other. Since random $K$ can be simulated without any knowledge about $A$ or $B$, pseudorandomness implies that the following 4 distributions are indistinguishable from each other:

- (real, real, random),
- (real, random, random),
- (random, real, random),
- (random, random, random).

Security says that (real, real, random) is indistinguishable from (real, real, real), so now we have 5 indistinguishable distributions under pseudorandomness plus security. Furthermore, (random, real, real) is indistinguishable from (real, real, real); this is because a reduction to first pseudorandomness can sample $b$ on its own and simulate both real $B$ and real $K$ without knowing $a$. (This immediately implies (random, real, real) is indistinguishable from (random, random,

random).) In summary, 6 out of the 7 distributions are indistinguishable from each other.

What about the last distribution, (real, random, real)? Suppose we attempt to construct a reduction that shows the indistinguishability from (real, real, real). The reduction, on (real $A$, real $B$) or (real $A$, random $B$), needs to simulate real $K$—which it cannot do because it knows *neither* $a$ (which is used in real $A$) *nor* $b$ (which is used in real $B$). What we need here is that (real $A$, real $B$) and (real $A$, random $B$) are indistinguishable *even given real* $K$, which is exactly *strong* second pseudorandomness.

In summary, security plus strong pseudorandomness imply the indistinguishability between all 7 distributions of $(A, B, K)$ (which in particular implies (real, random, real) is indistinguishable from (real, random, random)); whereas security plus pseudorandomness only imply the indistinguishability between 6 distributions of $(A, B, K)$, with (real, random, real) excluded.

*Counterexample.* We now present a concrete counterexample to show that security and pseudorandomness combined indeed do not imply the indistinguishability between (real, random, real) and the other 6 distributions. Consider a variant of hashed Diffie-Hellman, where

$$\begin{aligned} \mathsf{msg}_1(a) &= g^a \\ \mathsf{msg}_2(b, A) &= (g^b, H_0(A^b)) \\ \mathsf{key}_2(b, A) &= H_1(A^b) \end{aligned}$$

(where $H_0, H_1 \colon \mathbb{G} \to \{0,1\}^\kappa$ are ROs). In this variant, $P$ (that holds $a$) can use the $H_0$ hash to check whether it received a valid response to $A$; we use this to break strong pseudorandomness. Let

$$\mathsf{key}_1(a, (B_0, B_1)) = \begin{cases} H_1(B_0^a) & \text{if } B_1 = H_0(B_0^a) \\ H_2(g^a) & \text{otherwise} \end{cases}$$

Correctness is easily verified. Security and pseudorandomness still hold, as $g^a$ and $g^b$ are uniform elements of $\mathbb{G}$, and $H_0(A^b)$ and $H_1(A^b)$ are indistinguishable from random strings assuming CDH. However, (real, random, real) and (random, random, random) are easily distinguishable: in the former distribution, $K = H_2(A)$ with overwhelming probability because $B_1$ is uniform and so has negligible chance of satisfying $B_1 = H_0(B_0^a)$, while in the latter $K \neq H_2(A)$ with overwhelming probability, as they are independently random $\kappa$-bit strings.

*Necessity of Strong Pseudorandomness in EKE.* Consider the following simple attack on EKE (using IC): the adversary disregards $P'$, and on $\phi$ from $P$ sends random $\phi^*$ to $P$. After $P$ outputs $K$, the adversary queries $A := \mathcal{D}(\mathsf{pw}, \phi)$ and $B := \mathcal{D}(\mathsf{pw}, \phi^*)$ (where $\mathsf{pw}$ is the password of $P$).

In the real world, $A$ is the KA message generated by the honest $P$, so $A$ is real; $B$ is the IC decryption of random $\phi^*$, so $B$ is random; and $K$ is computed by the honest $P$ on $a$ and $B$, so $K$ is real. In other words, the environment's

view is (real, random, real). Now consider the ideal world: before $P$ outputs $K$ the simulator only sees a random $\phi^*$ from the adversary and has no knowledge about pw, so it cannot send a correct TestPwd command and thus $\mathcal{F}_{\text{PAKE}}$ will set the session key of $P$ to be random. (The simulator sees pw *after the session of* $P$ *completes*, at which time it cannot do anything to change the session key of $P$.) That is, in the ideal world the environment's view is ($\star$, $\star$, random), where $\star$ could be real or random, depending on the simulator's strategy. However, we have just seen that without strong pseudorandomness (real, random, real) is distinguishable from *all other 6 distributions*, so no matter what the two $\star$ are, the ideal world and the real world are distinguishable (the environment simply runs the distinguisher between (real, random, real) and the appropriate ($\star$, $\star$, random) for KA).

(For a diagram of the attack see [JRX24].)

The above shows that [MRR20, Theorem 10] is false in a way different from Sect. 3.2, since it claims the security of EKE without requiring strong pseudorandomness for KA. Furthermore, the issue persists even if $K$ is hashed, since security plus pseudorandomness do not even imply the unpredictability of real $K$ given real $A$ and random $B$ (as shown in the counterexample above). This suggests that [BCP+23, Theorem 1] is also false, since it does not require strong pseudorandomness. Also note that the attack above does not apply to OEKE, where the adversary needs to come up with a valid authenticator in order for $P$ to output a real session key.

### 3.4 Summary

We summarize the requirements on the underlying KA protocol for (O)EKE in Table 3. (For the attack on EKE using HIC/POPF, see [JRX24].)

**Table 3.** Requirements on the underlying KA protocol in (O)EKE

| | security and pseudorandomness | strong pseudorandomness | pseudorandom non-malleability | collision-resistance |
|---|---|---|---|---|
| EKE-PRF (or EKE if IC is used) | ✓ | ✓ (overlooked in [MRR20, BCP+23]) | ✓ (overlooked in [MRR20, SGJ23] and security analyses in [DHP+18, BCP+23]) | |
| OEKE | ✓ | | ✓ (overlooked in [SGJ23, BCP+23]) | ✓ (overlooked in security analysis of [SGJ23]) |
| EKE using HIC/POPF does not realize $\mathcal{F}_{\text{PAKE}}$ no matter what KA protocol is used, which is overlooked in [MRR20, SGJ23] | | | | |

In [JRX24] we argue that Kyber satisfies all of these properties, thus making it suitable for use in (O)EKE.

*How "real" are These Attacks?* It is fair to ask whether the attacks in this section correspond to "real-world" attacks, or if they merely break the UC PAKE security notion.

- The attack in Sect. 3.1 allows the adversary to predict a party's session key without knowing its password. Obviously, this completely breaks the security of PAKE.
- The attack in Sect. 3.3 breaks the *forward secrecy* of PAKE: the adversary sends a random message during the protocol session, but if it learns the password at any point after the session completes, the adversary can distinguish the party's session key from random (or even predict it). Forward secrecy is a standard requirement of modern key exchange, so this also constitutes a practical attack.
- The attack in Sect. 3.2 can be viewed as an attack on a generalized "per session" version of forward secrecy. Here, the adversary only learns the password during the second ($P'$-to-$P$) session, after the first ($P$-to-$P'$) session has already completed and $P'$ has output its session key. In this case the session key of $P'$ should be secure (because the adversary did not learn the password until after the session ended), yet the adversary can cause the two parties' session keys to be correlated.

## 4 Almost Universally Composable POPF

As mentioned in Sect. 1.2, the POPF definition in [MRR20] is unusual in requiring a higher-order security definition, saying that any weak PRF must remain secure when using the POPF as input. We now generalize this idea significantly to UC protocols, by defining a notion of *almost UC* realization, which means that a protocol must be composable with some class of protocols built on top.

**Definition 4.1.** Let $\mathcal{F}$ be an ideal functionality and $\mathcal{P}$ be a set of tuples of functionalities, protocols, and simulators. A protocol $\pi$ *$\mathcal{P}$-almost UC realizes* an ideal functionality $\mathcal{F}$ if, for every protocol $\rho$ that UC realizes an ideal functionality $\mathcal{F}'$ in the $\mathcal{F}$-hybrid model using a simulator[7] $\mathcal{S}$ such that $(\mathcal{F}', \rho, \mathcal{S}) \in \mathcal{P}$, the composed protocol $\rho^{\mathcal{F} \to \pi}$ (i.e., $\rho$ with $\mathcal{F}$ instantiated by $\pi$) UC realizes $\mathcal{F}'$.

If $\mathcal{P}$ contains every possible $(\mathcal{F}', \rho, \mathcal{S})$ then Def. 4.1 describes the standard notion of UC-realization, by the UC composition theorem.

### 4.1 The Functionality $\mathcal{F}_{\mathbf{POPF}}$

A POPF can be thought of as a family of random functions $\{F_\phi\}$, with two interfaces: Program, where a party picks a function $F_\phi$ with $F_\phi(x^*) := y^*$

[7] Technically, UC realization requires one simulator for every adversary. We only care about the simulator for the dummy adversary, which simply obeys whatever commands the environment gives to it.

Global variables (per $\mathsf{sid}$):
- Transcript $T = \{\}$ of POPF evaluations.
- Set $\Phi = \{\}$ of honest POPF indices.
- Malicious POPF index $\phi^* = \bot$.
- Random function $R \leftarrow \mathcal{Y}^{\{0,1\}^* \times \mathcal{X}}$ of uncontrollable outputs (defined via lazy sampling).
- String $\alpha^* = \bot$.

On $(\mathsf{Program}, \mathsf{sid}, x^*, y^*)$ from party $P$ (where $x \in \mathcal{X}$ and $y \in \mathcal{Y}$):
1. There are two cases, depending on whether there is an entry $(\cdot, x^*, y^*) \in T$.
   A. If there is no such entry, or if $P$ is malicious, send $(\mathsf{Program}, \mathsf{sid})$ to $\mathcal{A}^*$. Wait until $\mathcal{A}^*$ responds with $(\mathsf{Program}, \mathsf{sid}, \phi)$ such that there is no entry $(\phi, \cdot, \cdot) \in T$. Then add $\phi$ to $\Phi$.
   B. Otherwise, there is such an entry, and $P$ is honest. Send $(\mathsf{Program}, \mathsf{sid}, \{\phi \mid (\phi, x^*, y^*) \in T\})$ to $\mathcal{A}^*$. Wait until $\mathcal{A}^*$ responds with $(\mathsf{Program}, \mathsf{sid}, \phi)$ such that $(\phi, \cdot, \cdot) \notin T$ or $(\phi, x^*, y^*) \in T$.
2. If $(\phi, x^*, y^*) \notin T$, add $(\phi, x^*, y^*)$ to $T$.
3. Send $(\mathsf{Program}, \mathsf{sid}, \phi)$ to $P$.

On $(\mathsf{Eval}, \mathsf{sid}, \phi, x)$ from party $P$ (where $x \in \mathcal{X}$):
4. If $\phi \notin \Phi$, $\phi^* = \bot$, and $P$ is honest, then:
   (1) Send $(\mathsf{Extract}, \mathsf{sid}, \phi)$ to $\mathcal{A}^*$ and wait for response $(\mathsf{Extract}, \mathsf{sid}, x^*, \alpha)$.
   (2) Send $(\mathsf{Eval}, \mathsf{sid}, \phi, x^*)$ to $\mathcal{A}^*$ and wait for response $(\mathsf{Eval}, \mathsf{sid}, y^*)$.
   (3) Set $\phi^* := \phi$, $\alpha^* := \alpha$, and add $(\phi, x^*, y^*)$ to $T$.
5. Check if there is an entry $(\phi, x, y) \in T$. If not, generate $y$ according to three cases:
   A. If $\phi \in \Phi$, sample $y \leftarrow \mathcal{Y}$.
   B. If $\phi = \phi^*$, let $y := R(\alpha^*, x)$.
   C. Otherwise, send $(\mathsf{Eval}, \mathsf{sid}, \phi, x)$ to $\mathcal{A}^*$ and wait for response $(\mathsf{Eval}, \mathsf{sid}, y)$.
   Finally, add $(\phi, x, y)$ to $T$.
6. Send $(\mathsf{Eval}, \mathsf{sid}, y)$ to $P$.

On $(\mathsf{TestOutput}, \mathsf{sid}, \alpha, x)$ from $\mathcal{A}^*$ (where $\alpha \in \{0,1\}^*$ and $x \in \mathcal{X}$):
7. Send $(\mathsf{TestOutput}, \mathsf{sid}, R(\alpha, x))$ to $\mathcal{A}^*$.

**Fig. 4.** Ideal functionality $\mathcal{F}_{\mathrm{POPF}}$ (with domain $\mathcal{X}$ and range $\mathcal{Y}$).

for $(x^*, y^*)$ of its choice, and $\mathsf{Eval}$, where a party evaluates a function $F_\phi$ on a specific input $x$. Crucially, every honest function $F_\phi$ can be programmed at only one point $(x^*, y^*)$, and for any $x \neq x^*$, $F_\phi$ is random. Furthermore, POPFs must satisfy the *uncontrollable output* property: for an adversarially generated $\phi^*$, the output of $F_{\phi^*}(x)$ is pseudorandom—in particular, it is a suitable input of some higher-level schemes which use random inputs (e.g., a weak PRF, as in the definition in [MRR20])—except on a single $x$ "extracted" during the evaluation of $F_{\phi^*}(x)$.

Our POPF ideal functionality, $\mathcal{F}_{\mathrm{POPF}}$, is shown in Fig. 4. We now define POPFs using it and our almost UC definition above.

**Definition 4.2.** A protocol $\pi$ is a POPF if it $\mathcal{P}$-almost UC realizes $\mathcal{F}_{\mathrm{POPF}}$. Here, $\mathcal{P}$ is the set of all $(\mathcal{F}', \rho, \mathcal{S})$ such that $\mathcal{S}$ emulates $\mathsf{TestOutput}$ in the same way as

$\mathcal{F}_{\text{POPF}}$. That is, $\mathcal{S}$ must (lazily) sample a random function $R \leftarrow \mathcal{Y}^{\{0,1\}^\kappa \times \mathcal{X}}$, use it to emulate $(\mathsf{TestOutput}, \mathsf{sid}, \alpha, x)$ queries by returning $R(\alpha, x)$, and only make read-only oracle queries to $R$ (without programming $R$ in any way).

We require Def. 4.2 to ensure the simulator for the higher-level protocol does not program $R$, since the simulator for the POPF must program $R$ in order for $R$ to match evaluations of the POPF.

$\mathcal{F}_{\text{POPF}}$ maintains a set $T$ of defined function values, where $(\phi, x, y) \in T$ means that $F_\phi(x) = y$. When party $P$ (either honest or malicious) wants to program on a pair $(x^*, y^*)$, $\mathcal{F}_{\text{POPF}}$ lets the adversary specify a function index $\phi$. In particular, if there are no functions $F_\phi$ in the family such that $F_\phi(x^*)$ is set to be $y^*$, the adversary must choose a *new* index $\phi$; this ensures that $F_\phi$ is never programmed on two distinct points. $\mathcal{F}_{\text{POPF}}$ also adds $\phi$ to the set of honest POPF indices $\Phi$, which indicates that $F_\phi$ is "programmable-once" and has been programmed on one input/output pair. On the other hand, if there are such functions $F_\phi$ such that $F_\phi(x^*) = y^*$ (and $P$ is honest), $\mathcal{F}_{\text{POPF}}$ lets the adversary know all such indices $\phi$.Then the adversary has two options: either pick one of these existing indices (i.e., the programming process is identical to a previous one), or choose a *new* index $\phi$, just as in the previous case.

When a party $P$ wants to evaluate $F_\phi(x)$, whose result has not been defined through $T$, $\mathcal{F}_{\text{POPF}}$ has several cases based on how $\phi$ was generated.

- If $\phi$ is an honest index, this means that $F_\phi$ is "programmable-once" and has already been programmed on another input/output pair, so it chooses $F_\phi(x)$ at random.
- If $\phi$ is the "designated malicious index" $\phi^*$, we want to use $F_\phi(x)$ as the input of some higher-level scheme. The adversary should be able to learn $F_\phi(x)$, but not control it when $x$ differs from its chosen target $x^*$. Therefore, we set $F_\phi(x)$ to be the output of a random function $R$, queried on $\alpha$ and $x$. ($\alpha$ allows for the partial control of $F_\phi(x)$ given by rejection sampling on $\phi$.)
- Otherwise, i.e., if $\phi$ is a malicious index other than $\phi^*$, $\mathcal{F}_{\text{POPF}}$ allows the adversary to program $F_\phi$ on all inputs; in particular, $\mathcal{F}_{\text{POPF}}$ asks the adversary for $F_\phi(x)$.

The "designated malicious index" $\phi^*$ is chosen as the first malicious POPF index $\phi^*$ given to $\mathsf{Eval}$ by an honest party. Having only a single designated target is a necessary limitation of our POPF construction—requiring that multiple POPFs have jointly uncontrollable outputs is much more stringent than just requiring a single POPF to be uncontrollable. In all existing protocols that use POPFs, they only need to be individually uncontrollable and not jointly uncontrollable, so this limitation is not too onerous.

### 4.2 POPF Construction

Our POPF construction is identical to the 2-round Feistel POPF in [MRR20]. It uses an abelian group $\mathbb{G}$ and two ROs $H : \{0,1\}^* \times \{0,1\}^{3\kappa} \to \mathbb{G}$ and $H' :$

$\{0,1\}^* \times \mathbb{G} \to \{0,1\}^{3\kappa}$. The function family is indexed by $\phi = (s,t) \in \{0,1\}^{3\kappa} \times \mathbb{G}$ and defined as

$$F_\phi(x) = H(x, s \oplus H'(x,t)) \cdot t.$$

To program on an input/output pair $(x^*, y^*)$, one can compute $\phi = (s,t)$ via the following process: sample $r \leftarrow \{0,1\}^{3\kappa}$, and solve the equations

$$\begin{cases} H(x^*, r) \cdot t = y^*, \\ s \oplus H'(x^*, t) = r \end{cases}$$

for first $t$, then $s$. Note that for a pair of $(x^*, y^*)$, there are exponentially many $\phi$ such that $F_\phi(x^*) = y^*$. This is a crucial difference from the ideal cipher, where the encryption is deterministic.

The formal description of our construction is shown in Fig. 5.

On $(\mathsf{Program}, \mathsf{sid}, x^*, y^*)$ (where $x^* \in \mathcal{X}$ and $y^* \in \mathbb{G}$):
1. Choose $r \leftarrow \{0,1\}^{3\kappa}$.
2. Query $h := H(x^*, r)$, and compute $t := y^*/h$.
3. Query $h' := H'(x^*, t)$, and compute $s := r \oplus h'$.
4. Output $(\mathsf{Program}, \mathsf{sid}, (s,t))$.

On $(\mathsf{Eval}, \mathsf{sid}, (s,t), x)$ (where $x \in \mathcal{X}$, $s \in \{0,1\}^{3\kappa}$, and $t \in \mathbb{G}$):
5. Query $h' := H'(x,t)$, and compute $r := s \oplus h'$.
6. Query $h := H(x,r)$, and compute $y := h \cdot t$.
7. Output $(\mathsf{Eval}, \mathsf{sid}, y)$.

**Fig. 5.** POPF construction $\pi_{\mathrm{POPF}}$ (with domain $\mathcal{X}$ and range $\mathbb{G}$).

### 4.3 Security Analysis

**Theorem 4.1.** *The protocol $\pi_{\mathrm{POPF}}$ is a POPF. That is, for any protocol $\rho$ that UC realizes an ideal functionality $\mathcal{F}'$ in the $\mathcal{F}_{\mathrm{POPF}}$-hybrid model, where the simulator emulates* TestOutput *and $R$ in the same way as $\mathcal{F}_{\mathrm{POPF}}$, the composed protocol $\rho^{\mathcal{F} \to \pi}$ UC realizes $\mathcal{F}'$.*

The proof of Thm. 4.1 can be found in [JRX24].

## 5 PAKE Protocols Based on POPF

In this section we present our main results, namely the UC-security analysis of EKE and OEKE using POPF. Concretely,

- In Sect. 5.2 we prove that EKE-PRF using POPF is UC-secure assuming the underlying KA protocol satisfies security, strong pseudorandomness, and pseudorandom non-malleability. Additional results about the "raw" version of EKE, including (1) EKE using IC is UC-secure, and (2) EKE using POPF realizes the weaker "PAKE with same password test" functionality $\mathcal{F}_{\text{PAKE-sp}}$ (both results are under the same assumptions on KA), are argued in [JRX24].
- In Sect. 5.3 we prove that OEKE using POPF is UC-secure assuming the underlying KA protocol satisfies security, pseudorandomness, pseudorandom non-malleability, and collision resistance. This covers the OEKE-PRF and OEKE-RO variants in existing works.

Our starting observation is that the first round of EKE and OEKE are exactly identical. To make our security proofs more concise and modular, we abstract the first round into an ideal functionality $\mathcal{F}_{\text{EKE-1r}}$, and prove that the first round of (O)EKE realizes $\mathcal{F}_{\text{EKE-1r}}$; after that, we prove separately that the EKE and OEKE are secure in the $\mathcal{F}_{\text{EKE-1r}}$-hybrid world.

### 5.1 The First-Round Functionality and Protocol

*The Functionality.* See Fig. 6 for the UC functionality $\mathcal{F}_{\text{EKE-1r}}$ representing the first-round of (O)EKE, parameterized by a specific underlying KA protocol $\mathsf{KA}$. (Although we name the functionality $\mathcal{F}_{\text{EKE-1r}}$, we stress that the first round of OEKE is represented by the same functionality.)

Parameters: KA protocol $\mathsf{KA} = (\mathsf{msg}_1, \mathsf{msg}_2, \mathsf{key}_1, \mathsf{key}_2)$.

- On input $(\mathsf{Program}, \mathsf{sid}, P, P', \mathsf{pw}, A)$ from $P$, send $(\mathsf{Program}, \mathsf{sid}, P, P')$ to $\mathcal{S}$. If this is the first $\mathsf{Program}$ message for $\mathsf{sid}$, then record $\langle \mathsf{Program}, P, P', \mathsf{pw}, A \rangle$.
- On input $(\mathsf{SampleResp}, \mathsf{sid}, P', P, \mathsf{pw}')$ from $P'$, send $(\mathsf{SampleResp}, \mathsf{sid}, P', P)$ to $\mathcal{S}$. If this is the first $\mathsf{SampleResp}$ message for $\mathsf{sid}$, then sample $b \leftarrow \mathcal{R}$ and record $\langle \mathsf{SampleResp}, P', P, \mathsf{pw}', b \rangle$.
- On $(\mathsf{Eval}, \mathsf{sid}, P, P', x)$ from $\mathcal{S}$, send $A$ to $\mathcal{S}$, where $A$ is defined as follows:
  - If there is a record $\langle \mathsf{Program}, P, P', \mathsf{pw}, A' \rangle$ with $\mathsf{pw} = x$, set $A := A'$.
  - Otherwise if there is a record $\langle \mathsf{Eval}, \mathsf{pw}, A'' \rangle$ with $\mathsf{pw} = x$, set $A := A''$.
  - Otherwise sample $a \leftarrow \mathcal{R}$ and set $A := \mathsf{msg}_1(a)$. Record $\langle \mathsf{Eval}, x, A \rangle$.
- On $(\mathsf{Deliver}, \mathsf{sid}, P, P', \mathsf{pw}^*, A^*)$ from $\mathcal{S}$, if there is a record $\langle \mathsf{SampleResp}, P', P, \mathsf{pw}', b \rangle$, and this is the first $\mathsf{Deliver}$ message for $\mathsf{sid}$, then output $(\mathsf{sid}, B, K')$ to $P'$, where $B$ and $K'$ are defined as follows:
  1. If $\mathsf{pw}^* = \perp$ and there is a record $\langle \mathsf{Program}, P, P', \mathsf{pw}, A \rangle$, overwrite $\mathsf{pw}^* := \mathsf{pw}$ and $A^* := A$.
  2. Next, if $\mathsf{pw}^* = \mathsf{pw}'$, then set $B := \mathsf{msg}_2(b, A^*)$ and $K' := \mathsf{key}_2(b, A^*)$. Else, sample $B \leftarrow \mathcal{M}_2$ and $K' \leftarrow \mathcal{K}$.

**Fig. 6.** Ideal functionality $\mathcal{F}_{\text{EKE-1r}}$ representing the first round of (O)EKE.

Recall that in the first round of both EKE and OEKE, party $P$ (with password $\mathsf{pw}$) computes its KA message $A$, programs $\phi$ such that $A = F_\phi(\mathsf{pw})$, and

sends $\phi$ to $P'$. Then $P'$ (with password $\mathsf{pw}'$) evaluates $A' = F_\phi(\mathsf{pw}')$, samples randomness $b \leftarrow \mathcal{R}$, sends $B := \mathsf{msg}_2(b, A')$ to $P$, and outputs $K' := \mathsf{key}_2(b, A')$. (Note that the randomness $a$ that corresponds to $A$ is not used in the first round.) The man-in-the-middle adversary has the following capacity:

- The adversary can evaluate $F_\phi(x)$ for any (fresh) $x$ of its choice. If $x = \mathsf{pw}$ (which is the point at which $F_\phi$ is programmed) the result is $A$, otherwise the result is a random value.
- The adversary may or may not modify the $P$-to-$P'$ message $\phi$:
  - Suppose the adversary does not modify the message. Then if $\mathsf{pw} = \mathsf{pw}'$ (i.e., the passwords of $P$ and $P'$ match), $P'$ will send $B = \mathsf{msg}_2(b, A)$ to $P$ and output $K' = \mathsf{key}_2(b, A)$; if $\mathsf{pw} \neq \mathsf{pw}'$, $P'$ will send $B = \mathsf{msg}_2(b, A')$ and output $K' = \mathsf{key}_2(b, A')$ where $A' = F_\phi(\mathsf{pw}')$ is a random value, so $B$ and $K'$ are pseudorandom.
  - Suppose the adversary modifies the message $\phi$ to some other $\phi^*$, which incorporates a password guess $\mathsf{pw}^*$. Let $A^* = F_{\phi^*}(\mathsf{pw}^*)$; if $\mathsf{pw}^* = \mathsf{pw}'$ (i.e., the password guess is correct), $P'$ will send $B = \mathsf{msg}_2(b, A^*)$ to $P$ and output $K' = \mathsf{key}_2(b, A^*)$, and the adversary may compute $K'$ as $\mathsf{key}_1(a^*, B)$ (where $a^*$ is the randomness corresponding to $A^*$). If $\mathsf{pw}^* \neq \mathsf{pw}'$, $P'$ will send $B = \mathsf{msg}_2(b, A')$ and output $K' = \mathsf{key}_2(b, A')$ where $A' = F_{\phi^*}(\mathsf{pw}')$ is a random value, so $B$ and $K'$ are pseudorandom.

In the real world, the evaluation of $F_\phi(x)$ is modeled by the Eval command that defines the answer $A$ just as in the ideal world, except that if $\mathsf{pw} \neq x$ the answer is the "real" $\mathsf{msg}_1(a)$ instead of random. For the password test modeled by the Deliver command, the ideal adversary $\mathcal{S}$ may specify a $\mathsf{pw}^*$ and an $A^*$, and there are three cases:

- $\mathsf{pw}^* = \perp$ models the real-world scenario where the adversary does not modify the $P$-to-$P'$ message. Then if $\mathsf{pw} = \mathsf{pw}'$, the functionality sets $B = \mathsf{msg}_2(b, A)$ and $K' = \mathsf{key}_2(b, A)$. (Concretely, the functionality enters step 1 and redefines $\mathsf{pw}^* := \mathsf{pw}$ and $A^* := A$, and then enters step 2, checks that $\mathsf{pw}^* = \mathsf{pw}'$, and sets $B = \mathsf{msg}_2(b, A^*)$ and $K' = \mathsf{key}_2(b, A^*)$.)
- $\mathsf{pw}^* \neq \perp$ models the real-world scenario where the adversary modifies the $P$-to-$P'$ message and incorporates a password guess $\mathsf{pw}^*$ for $P'$. If $\mathsf{pw}^* = \mathsf{pw}'$, the functionality sets $B = \mathsf{msg}_2(b, A^*)$ and $K' = \mathsf{key}_2(b, A^*)$.
- Otherwise (i.e., $\mathsf{pw}^* = \perp$ and $\mathsf{pw} \neq \mathsf{pw}'$; or $\mathsf{pw}^* \neq \perp$ and $\mathsf{pw}^* \neq \mathsf{pw}'$) the functionality samples $B$ and $K'$ at random.

*The Protocol.* See Fig. 7 for the first-round protocol. Recall that the first round of all (O)EKE variants we analyze in this work is identical.

*Security Analysis.*

**Lemma 5.1.** *Suppose that* KA *is a secure and pseudorandom KA protocol. Then the protocol in Fig. 7 realizes* $\mathcal{F}_{\text{EKE-1r}}$ *(Fig. 6) in the* $\mathcal{F}_{\text{POPF}}$*-hybrid world via a simulator* $\mathcal{S}$ *that emulates* TestOutput *in the same way as* $\mathcal{F}_{\text{POPF}}$*.*

We defer the proof of Lem. 5.1 to the full version [JRX24].

Parameters:
- POPF functionality $\mathcal{F}_{\text{POPF}}$ (see Figure 4) with domain $\{0,1\}^*$ and range $\mathcal{M}_1$.

On input $(\mathsf{Program}, \mathsf{sid}, P, P', \mathsf{pw}, A)$, party $P$ does the following:
1. Send $(\mathsf{Program}, \mathsf{sid}, \mathsf{pw}, A)$ to $\mathcal{F}_{\text{POPF}}$ and wait for response $(\mathsf{Program}, \mathsf{sid}, \phi)$.
2. Send $(\mathsf{sid}, \phi)$ to $P'$.

On input $(\mathsf{SampleResp}, \mathsf{sid}, P', P, \mathsf{pw}')$, party $P'$ samples $b \leftarrow \mathcal{R}$ and records $\langle \mathsf{pw}', b \rangle$.

On message $(\mathsf{sid}, \phi)$ from $P$, if there is a record $\langle \mathsf{pw}', b \rangle$ then party $P'$ does the following:
3. Send $(\mathsf{Eval}, \mathsf{sid}, \phi, \mathsf{pw}')$ to $\mathcal{F}_{\text{POPF}}$ and wait for response $(\mathsf{Eval}, \mathsf{sid}, A')$.
4. Compute $B := \mathsf{msg}_2(b, A')$ and $K' := \mathsf{key}_2(b, A')$.
5. Output $(\mathsf{sid}, B, K')$.

**Fig. 7.** (O)EKE first-round protocol, in the $\mathcal{F}_{\text{POPF}}$-hybrid world.

Parameters:
- EKE first round functionality $\mathcal{F}_{\text{EKE-1r}}$ (Figure 6) for a KA protocol KA.
- POPF functionality $\mathcal{F}_{\text{POPF}}$ (see Figure 4) with domain $\{0,1\}^*$ and range $\mathcal{M}_2$.
- A pseudorandom function $\mathsf{PRF}_K : \Phi \to \mathcal{K}$ with key space $\mathcal{K}$.

On input $(\mathsf{NewSession}, \mathsf{sid}, P, P', \mathsf{pw}, \text{“initiator”})$, party $P$ does the following:
1. Sample and record $a \leftarrow \mathcal{R}$, and compute $A := \mathsf{msg}_1(a)$.
2. Send $(\mathsf{Program}, sid, P, P', \mathsf{pw}, A)$ to $\mathcal{F}_{\text{EKE-1r}}$.

On input $(\mathsf{NewSession}, \mathsf{sid}, P', P, \mathsf{pw}', \text{“respondent”})$, party $P'$ sends $(\mathsf{SampleResp}, \mathsf{sid}, P, P', \mathsf{pw}')$ to $\mathcal{F}_{\text{EKE-1r}}$.

On message $(\mathsf{sid}, B, K')$ from $\mathcal{F}_{\text{EKE-1r}}$, party $P'$ does the following:
3. Send $(\mathsf{Program}, \mathsf{sid}, \mathsf{pw}', B)$ to $\mathcal{F}_{\text{POPF}}$ and wait for response $(\mathsf{Program}, \mathsf{sid}, \phi')$.
4. Send $(\mathsf{sid}, \phi')$ to $P$.
5. Output $(\mathsf{sid}, \mathsf{PRF}_{K'}(\phi'))$.

On message $(\mathsf{sid}, \phi')$ from $P'$, party $P$ does the following:
6. Send $(\mathsf{Eval}, \mathsf{sid}, \phi', \mathsf{pw})$ to $\mathcal{F}_{\text{POPF}}$ and wait for response $(\mathsf{Eval}, \mathsf{sid}, B)$.
7. Retrieve $a$ and compute $K := \mathsf{key}_1(a, B)$.
8. Output $(\mathsf{sid}, \mathsf{PRF}_K(\phi'))$.

**Fig. 8.** EKE-PRF protocol, in the $(\mathcal{F}_{\text{POPF}}, \mathcal{F}_{\text{EKE-1r}})$-hybrid world.

### 5.2 The EKE Protocol

**Theorem 5.1.** *Suppose that* KA *is a correct, secure, strongly pseudorandom, and pseudorandom non-malleable KA protocol. Then the EKE-PRF protocol (Fig. 8) realizes* $\mathcal{F}_{\text{PAKE}}$ *in the* $(\mathcal{F}_{\text{POPF}}, \mathcal{F}_{\text{EKE-1r}})$*-hybrid world via a simulator* $\mathcal{S}$ *that emulates* TestOutput *in the same way as* $\mathcal{F}_{\text{POPF}}$.[8]

[8] Applying a PRF while producing the session key is necessary for the protocol to realize $\mathcal{F}_{\text{PAKE}}$; for an explanation, see [JRX24].

**Table 4.** Summary of hybrids for EKE-PRF security. "$" denotes "chosen at random from respective range". $SK, SK'$ denote the outputs of $P, P'$ respectively.

| # | case addressed | change | property used |
|---|---|---|---|
| 1 | $\mathsf{pw} = \mathsf{pw}'$ and $\mathcal{A}$ eavesdrops or re-encrypts | $K := K'$ | KA correctness |
| 2 | $\mathsf{pw} = \mathsf{pw}' \wedge \mathsf{pw}^* = \bot$ | $B, K'$ \$ | KA pseudorandom non-malleability |
| 3 | $(\phi')^*$ contains a wrong password guess or $\mathsf{pw} \neq \mathsf{pw}' \wedge (\phi')^* = \phi'$ | $K$ \$ | (KA security / strong pseudorandomness) |
| 4 | $\mathsf{pw}^* \neq \bot \wedge \mathsf{pw} = \mathsf{pw}' \neq \mathsf{pw}^* \wedge (\phi')^* = \phi'$ | $K$ \$ | (KA security / strong pseudorandomness) |
| 5 | $\mathcal{A}$ eavesdrops | $SK := SK'$ | none |
| 6 | all cases except $\mathsf{pw}^* = \mathsf{pw}'$ | $\mathsf{PRF}_{K'}$ \$ | PRF property |
| 7 | cases in hybrids 3 and 4 | $\mathsf{PRF}_K$ \$ | PRF property |
| 8 | $\mathcal{A}$ re-encrypts | $SK := \mathsf{PRF}_K((\phi')^*)$ | PRF property |
| 9 | | $K := \mathsf{key}_1(a, B)$ | (KA security / strong pseudorandomness) |

The proof of Thm. 5.1 is given in [JRX24]. For a summary of the hybrids in the proof, see Table 4.

The theorem immediately implies the following: let KA be a correct, secure, strongly pseudorandom, and pseudorandom non-malleable KA protocol. Beginning with the protocol in Fig. 8, replace $\mathcal{F}_{\text{EKE-1r}}$ with the protocol in Fig. 7, and $\mathcal{F}_{\text{POPF}}$ with the protocol in Fig. 5. Then by Lem. 5.1 and Thms. 5.1 and 4.1 the resulting protocol realizes $\mathcal{F}_{\text{PAKE}}$ in the ROM.

**Remark 5.1.** Our security analysis of EKE-PRF critically relies on the fact that $P$ is the initiator and $P'$ is the responder, i.e., the message and session key of $P'$ depend on the message of $P$. If the underlying KA is 1-simultaneous round, and there is no mechanism ensuring that $P'$ always outputs first, then whether $\phi$ or $\phi'$ is the first message depends on the adversary who decides which one to deliver first. So in this case both messages need to be included in the PRF, i.e., the session key should be $\mathsf{PRF}_K(\phi, \phi')$.

### 5.3 The OEKE Protocol

**Theorem 5.2.** *Suppose that* KA *is a correct, pseudorandom non-malleable, and collision resistant KA protocol, whose key space* $\mathcal{K} = \{0,1\}^{3\kappa}$. *Then the OEKE protocol (Fig. 9) realizes* $\mathcal{F}_{\text{PAKE}}$ *in the* $\mathcal{F}_{\text{EKE-1r}}$*-hybrid world.*[9]

[9] Our protocol realizing $\mathcal{F}_{\text{EKE-1r}}$ (Fig. 7) additionally requires KA to be secure and pseudorandom.

Parameters:
– EKE first round functionality $\mathcal{F}_{\text{EKE-1r}}$ (see Figure 6) for a KA protocol $\mathsf{KA}$.

On input $(\mathsf{NewSession}, \mathsf{sid}, P, P', \mathsf{pw}, \text{“initiator”})$, party $P$ does the following:
1. Sample and record $a \leftarrow \mathcal{R}$, and compute $A := \mathsf{msg}_1(a)$.
2. Send $(\mathsf{Program}, sid, P, P', \mathsf{pw}, A)$ to $\mathcal{F}_{\text{EKE-1r}}$.

On input $(\mathsf{NewSession}, \mathsf{sid}, P', P, \mathsf{pw}', \text{“respondent”})$, party $P'$ sends $(\mathsf{SampleResp}, sid, P, P', \mathsf{pw}')$ to $\mathcal{F}_{\text{EKE-1r}}$.

On message $(\mathsf{sid}, B, K' \,\|\, \tau')$ from $\mathcal{F}_{\text{EKE-1r}}$, party $P'$ does the following:
3. Send $(\mathsf{sid}, B, \tau')$ to $P$.
4. Output $(\mathsf{sid}, K')$.

On message $(\mathsf{sid}, B, \tau')$ from $P'$, party $P$ does the following:
5. Retrieve $a$ and compute $K \,\|\, \tau := \mathsf{key}_1(a, B)$.
6. Check if $\tau = \tau'$. If so, output $(\mathsf{sid}, K)$. Otherwise output $(\mathsf{sid}, K_\$)$ where $K_\$ \leftarrow \{0,1\}^\kappa$.

**Fig. 9.** OEKE protocol, in the $\mathcal{F}_{\text{EKE-1r}}$-hybrid world.

**Table 5.** Summary of hybrids for OEKE security. “$” denotes “chosen at random from respective range”.

| # | case addressed | change | property used |
|---|---|---|---|
| 1 | $\mathsf{pw}^* = \bot \wedge \mathsf{pw} = \mathsf{pw}'$ $\wedge B^* = B$ | $K = K'$ if $\tau^* = \tau'$<br>$K$ \$ otherwise | KA correctness |
| 2 | $\mathsf{pw}^* = \bot \wedge \mathsf{pw} = \mathsf{pw}'$ | $K', \tau'$ \$ | KA pseudorandom non-malleability |
| 3 | $\mathsf{pw}^* = \bot \wedge \mathsf{pw} \neq \mathsf{pw}'$ $\wedge B^* = B \wedge \tau^* = \tau'$ | $K$ \$ | none |
| 4 | $\mathsf{pw}^* \neq \bot$ $\vee B^* \neq B \vee \tau^* \neq \tau'$ | exclude collisions<br>while extracting “good” $(\mathsf{pw}')^*$ | KA collision resistance |
| 5 | | $K = \mathsf{key}_1(a^*, B^*)[1]$ if $\mathsf{pw}$ “good”<br>$K$ \$ otherwise | KA collision resistance |
| 6 | | extract “good” $(\mathsf{pw}')^*$<br>$K = \mathsf{key}_1(a^*, B^*)[1]$ if $(\mathsf{pw}')^* = \mathsf{pw}$<br>$K$ \$ if $(\mathsf{pw}')^* \neq \mathsf{pw}$ or no $(\mathsf{pw}')^*$ | none |

The proof of Thm. 5.2 is given in [JRX24]. For a summary of the hybrids in the proof, see Table 5.

The theorem immediately implies the following: let $\mathsf{KA}$ be a correct, secure, pseudorandom, pseudorandom non-malleable, and collision-resistant KA protocol. Beginning with the protocol in Fig. 9, replace $\mathcal{F}_{\text{EKE-1r}}$ with the protocol in Fig. 7, and $\mathcal{F}_{\text{POPF}}$ with the protocol in Fig. 5. Then by Lem. 5.1 Thms. 5.2 and 4.1 the resulting protocol realizes $\mathcal{F}_{\text{PAKE}}$ in the ROM.

*OEKE-PRF and OEKE-RO.* Since our Thm. 5.2 considers a general OEKE protocol with *any* KA protocol whose key length is $3\kappa$, this immediately covers OEKE-PRF as a special case, where the KA protocol is obtained via taking another KA protocol whose key length is $\kappa$ and applying a PRF to the key (on inputs such as 0, 1 and 2). OEKE-RO is in turn a special case of OEKE-PRF, where the PRF is an RO.

## References


[ABB+20] Abdalla, M., Barbosa, M., Bradley, T., Jarecki, S., Katz, J., Xu, J.: Universally composable relaxed password authenticated key exchange. In: Micciancio, D., Ristenpart, T. (eds.) CRYPTO 2020. LNCS, vol. 12170, pp. 278–307. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-56784-2_10

[ABJS24] Arriaga, A., Barbosa, M., Jarecki, S., Skrobot, M.: C'est Très CHIC: a compact password-authenticated key exchange from lattice-based KEM. In: Chung, K.-M., Sasaki, Yu. (eds.) ASIACRYPT 2024. Part V, volume 15488 of LNCS, pp. 3–33. Springer, Singapore (2024). https://doi.org/10.1007/978-981-96-0935-2_1

[AHH21] Abdalla, M., Haase, B., Hesse, J.: Security analysis of CPace. In: Tibouchi, M., Wang, H. (eds.) ASIACRYPT 2021. LNCS, vol. 13093, pp. 711–741. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-92068-5_24

[AP05] Abdalla, M., Pointcheval, D.: Simple password-based encrypted key exchange protocols. In: Menezes, A. (ed.) CT-RSA 2005. LNCS, vol. 3376, pp. 191–208. Springer, Heidelberg (2005). https://doi.org/10.1007/978-3-540-30574-3_14

[AWZ23] Abram, D., Waters, B., Zhandry, M.: Security-preserving distributed samplers: How to generate any CRS in one round without random oracles. In: Handschuh, H., Lysyanskaya, A. (eds.) CRYPTO 2023. Part I. LNCS, vol. 14081, pp. 489–514. Springer, Heidelberg (2023). https://doi.org/10.1007/978-3-031-38557-5_16

[BCP03] Bresson, E., Chevassut, O., Pointcheval, D.: Security proofs for an efficient password-based key exchange. In: Jajodia, S., Atluri, V., Jaeger, T. (eds.) ACM CCS 2003, pp. 241–250 (2). ACM Press003

[BCP+23] Beguinet, H., Chevalier, C., Pointcheval, D., Ricosset, T., Rossi, M.: GeT a CAKE: generic transformations from key encaspulation mechanisms to password authenticated key exchanges. In: Tibouchi, M., Wang, X. (eds.) ACNS 23. Part II, LNCS, vol. 13906, pp. 516–538. Springer, Heidelberg (2023). https://doi.org/10.1007/978-3-031-33491-7_19

[BM92] Bellovin, S.M., Merritt, M.: Encrypted key exchange: password-based protocols secure against dictionary attacks. In: 1992 IEEE Symposium on Security and Privacy, pp. 72–84. IEEE Computer Society Press (1992)

[BPR00] Bellare, M., Pointcheval, D., Rogaway, P.: Authenticated key exchange secure against dictionary attacks. In: Preneel, B. (ed.) EUROCRYPT 2000. LNCS, vol. 1807, pp. 139–155. Springer, Heidelberg (2000). https://doi.org/10.1007/3-540-45539-6_11

[Can01] Canetti, R.: Universally composable security: a new paradigm for cryptographic protocols. In: 42nd FOCS, pp. 136–145. IEEE Computer Society Press (2001)

[CDG+18] Camenisch, J., Drijvers, M., Gagliardoni, T., Lehmann, A., Neven, G.: The wonderful world of global random oracles. In: Nielsen, J.B., Rijmen, V. (eds.) EUROCRYPT 2018. LNCS, vol. 10820, pp. 280–312. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-78381-9_11

[CHK+05] Canetti, R., Halevi, S., Katz, J., Lindell, Y., MacKenzie, P.D.: Universally composable password-based key exchange. In: Cramer, R. (ed.) EUROCRYPT 2005. LNCS, vol. 3494, pp. 404–421. Springer, Heidelberg (2005). https://doi.org/10.1007/11426639_24

[Cry20] Crypto forum research group: PAKE selection (2020). https://github.com/cfrg/pake-selection

[DFG+23] Gareth, T.: Security analysis of the WhatsApp end-to-end encrypted backup protocol. In: Handschuh, H., Lysyanskaya, A. (eds.) CRYPTO 2023. Part IV, volume 14084 of LNCS, pp. 330–361. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-38551-3_11

[DHP+18] Dupont, P.-A., Hesse, J., Pointcheval, D., Reyzin, L., Yakoubov, S.: Fuzzy password-authenticated key exchange. In: Nielsen, J.B., Rijmen, V. (eds.) EUROCRYPT 2018. LNCS, vol. 10822, pp. 393–424. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-78372-7_13

[DS16] Dai, Y., Steinberger, J.: Indifferentiability of 8-round feistel networks. In: Robshaw, M., Katz, J. (eds.) CRYPTO 2016. LNCS, vol. 9814, pp. 95–120. Springer, Heidelberg (2016). https://doi.org/10.1007/978-3-662-53018-4_4

[GK10] Groce, A., Katz, J.: A new framework for efficient password-based authenticated key exchange. In: Al-Shaer, E., Keromytis, A.D., Shmatikov, V., (eds.) ACM CCS 2010, pp. 516–525 (2010). ACM Press

[GL03] Gennaro, R., Lindell, Y.: A framework for password-based authenticated key exchange. In: Biham, E. (ed.) EUROCRYPT 2003. LNCS, vol. 2656, pp. 524–543. Springer, Heidelberg (2003). https://doi.org/10.1007/3-540-39200-9_33

[HL19] Haase, B., Labrique, B.: AuCPace: efficient verifier-based PAKE protocol tailored for the IIoT. In: IACR Transactions on Cryptographic Hardware and Embedded Systems, pp. 1–48 (2019)

[Jar23] Jarecki, S.: Randomized half-ideal cipher on groups with applications to UC (a)PAKE (2023). https://www.youtube.com/watch?v=GL4m7StDsPg, 2023. Talk at EUROCRYPT 2023

[JKX18] Jarecki, S., Krawczyk, H., Xu, J.: OPAQUE: an asymmetric PAKE protocol secure against pre-computation attacks. In: Nielsen, J.B., Rijmen, V. (eds.) EUROCRYPT 2018. LNCS, vol. 10822, pp. 456–486. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-78372-7_15

[JRX24] Januzelli, J., Roy, L., Xu, J.: Under what conditions is encrypted key exchange actually secure? Cryptology ePrint Archive, Paper 2024/324 (2024)

[KV11] Katz, J., Vaikuntanathan, V.: Round-optimal password-based authenticated key exchange. In: Ishai, Y. (ed.) TCC 2011. LNCS, vol. 6597, pp. 293–310. Springer, Heidelberg (2011). https://doi.org/10.1007/978-3-642-19571-6_18

[LLHG23] Liu, X., Liu, S., Han, S., Dawu, G.: EKE meets tight security in the Universally Composable framework. In: Boldyreva, A., Kolesnikov, V. (eds.) PKC 2023. Part I, vol. 13940 of LNCS, pp. 685–713. Springer, Heidelberg (2023). https://doi.org/10.1007/978-3-031-31368-4_24

[MRR20] McQuoid, I., Rosulek, M., Roy, L.: Minimal symmetric PAKE and 1-out-of-N OT from programmable-once public functions. In: Ligatti, J., Ou, X., Katz, J., Vigna, G., (eds.) ACM CCS 2020, pp. 425–442 (2020). ACM Press

[RX23] Roy, L., Xu, J.: A universally composable PAKE with zero communication cost. In: Boldyreva, A., Kolesnikov, V. (eds.) Public-Key Cryptography – PKC 2023. PKC 2023. LNCS, vol. 13940. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-31368-4_25

[SGJ23] Santos, B.F.D., Gu, Y., Jarecki, S.: Randomized half-ideal cipher on groups with applications to UC (a)PAKE. In: Hazay, C., Stam, M. (eds.) Advances in Cryptology – EUROCRYPT 2023. EUROCRYPT 2023. LNCS, vol. 14008. Springer, Cham (2023). https://doi.org/10.1007/978-3-031-30589-4_5

[Xag22] Xagawa, K.: Anonymity of NIST PQC round 3 KEMs. In: Dunkelman, O., Dziembowski, S. (eds.) Advances in Cryptology – EUROCRYPT 2022. EUROCRYPT 2022. LNCS, vol. 13277. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-07082-2_20